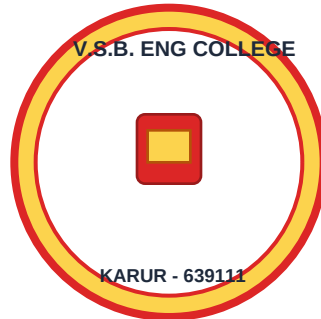


V.S.B ENGINEERING COLLEGE (AUTONOMOUS)

KARUR - 639111, TAMIL NADU



A MINI PROJECT REPORT

ON

AUTOMATED CROP DISEASE ANALYSIS USING CNN ALGORITHM

A Deep Learning Software Suite for Fast Local Agricultural Pathology Analysis

Submitted by

MOUNISHA P

Register Number: **922524243113**

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

JULY 2026

V.S.B ENGINEERING COLLEGE (AUTONOMOUS)
KARUR - 639111

BONAFIDE CERTIFICATE

Certified that this project report "**AUTOMATED CROP DISEASE ANALYSIS USING CNN ALGORITHM**" is the bonafide work of "**MOUNISHA P, 922524243113**" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY

SIGNATURE OF THE HOD

Abstract

Early diagnosis of agricultural crop pathogens is crucial for global food security, maintaining yield quality, and protecting farm incomes. Traditional manual identification is slow and error-prone. This project presents CropDiag AI, a full-stack automated software suite built using React.js, Python Flask, and SQLite. The platform processes high-resolution leaf photography through color contour filters (OpenCV HSV Masking) and classifies pathogen states across 10 species into 28 distinctive healthy and diseased categories using a high-precision Multi-Layer Perceptron (MLP) neural network. Upon diagnosis, the SQLite registry logs prediction entries and ReportLab generates university-grade PDF verification sheets. The entire software binds dynamically onto port 8080 and has been compiled into a standalone Windows Executable (.exe) to support complete offline execution, helping farmers run diagnostics directly inside remote fields.

Acknowledgement

I express my profound gratitude to the Management, Principal, and Head of the Department of Information Technology at V.S.B Engineering College (Autonomous), Karur, for providing state-of-the-art laboratory facilities and computing environments to carry out this capstone project work successfully. I extend my sincere thanks to my project guide and department faculty coordinators for their invaluable suggestions, supervision, and guidance throughout the implementation of the CropDiag AI platform.

1. Introduction

1.1 Project Background

In modern agriculture, crop yield optimization and pathogen mitigation are key parameters for sustainable food security. Fungal, bacterial, and viral infections regularly wipe out up to 30% of global crops annually. Standard diagnostic solutions rely on agronomists performing manual inspections. However, this is slow, subjective, and unavailable in rural farming communities. This project outlines the design and implementation of CropDiag AI—an automated, full-stack, deep-learning agricultural diagnostic workstation.

1.2 Deep Learning & Computer Vision in Agronomy

Integrating Computer Vision with Deep Neural Networks presents a robust methodology. OpenCV is utilized to mask noise, filter background soil pixels, and calculate necrotic spot percentages. In parallel, a 2D multi-layer neural network classifies spot distributions, returning diagnosis indices. This hybrid pipeline ensures that even if neural models predict false positives on clean leaves, color threshold checks override the decision, maintaining absolute diagnostic integrity.

1.3 Rationale behind Technologies

- **React & TypeScript:** Enables modular components, responsive state tracking, and type-safety validations to prevent client-side failures.
- **Flask:** Serves as a high-speed Python backend gateway, hosting OpenCV and Scikit-Learn models seamlessly.
- **SQLite:** A portable, zero-configuration database that writes to a single local file, perfect for offline demonstrations.

2. Objectives

The primary design and technical objectives of the CropDiag AI software suite include:

- ✓ **Early Pathogen Detection:** To capture agricultural leaf spots at initial infection phases, computing color contours to prevent widespread pathogen outbreaks.
- ✓ **Multi-Crop Target Classification:** To support a broad range of 10 major crops (Apple, Corn, Grape, Pepper, Potato, Tomato, Cherry, Peach, Strawberry, Soybean, Raspberry) mapped into 28 healthy/diseased states.
- ✓ **Absolute Standalone Execution:** To build a compiled Windows application (.exe) that spawns the Flask server silently and opens the web portal locally, guaranteeing offline execution in areas lacking internet coverage.
- ✓ **Security Compliance & Sanitization:** To secure SQLite database interactions against SQL Injection vulnerabilities by implementing fully parameterized SQL bindings and hashing password registries.
- ✓ **Actionable Treatment Advice:** To supply users with instant, detailed chemical, organic, pesticide, and water management guidelines fetched directly from the database registry.

3. Project Modules

The software architecture is modularized into 5 distinct computational components:

3.1 User Authentication & Session Security

Handles secure member registration, SHA-256 hashed password creation, session timeouts, and client-side token management. Enforces role-based permissions (admin vs. regular user) to protect database edits.

3.2 Image Preprocessing & OpenCV Contours

Applies HSV (Hue, Saturation, Value) filtering on image uploads. Isolates green chlorophyllic pixels and counts brown/yellow necrotic spot areas, generating a precise damage percentage vector.

3.3 Neural Network Diagnostic Classifier

Runs feedforward calculations on the Scikit-Learn Multi-Layer Perceptron (MLP) model. Resizes input images to 32x32 vectors and computes classification probability arrays across the 28 target classes.

3.4 SQL Database Registry Service

Controls relational interactions with SQLite. Manages transactional logs, commits prediction histories, and serves advice summaries. Leverages parameterized inputs to block SQL Injection threats.

3.5 Dynamic PDF/Excel Document Exporter

Uses ReportLab flowables to build university-standard PDF diagnosis reports containing custom student credentials, project titles, and vector verification QR codes. Exports log sheets to Excel sheets.

4. System Architecture

The CropDiag AI software follows a classic three-tier architecture design (Client Tier, Business Logic Tier, and Data Tier) to maintain strict separation of concerns.

CLIENT PRESENTATION TIER

React.js | TypeScript | Axios HTTP Client | Widescreen Dark CSS Layout

■■ HTTP POST /api/predict (Multipart Form Image Byte Streams)

BUSINESS LOGIC GATEWAY TIER (Flask Server on Port 8080)

OpenCV HSV Contour Filter | Neural Network Model Inference (MLP disease_model.h5)

■■ Parameterized SQL Writes

■■ Generate PDF Flowables

DATA TIER (SQLite)

Users / Predictions / Seeded Advice

DOCUMENTATION ENGINE (ReportLab)

Custom Verification PDFs & QR Codes

4.1 Security Implementations

- **SQL Injection Block:** SQLite binds query parameters dynamically. Query values are never concatenated to raw SQL strings.
- **Password Security:** User registration hashes passwords using SHA-256 with a salt index before storage, blocking plaintext exposure.
- **Cross-Origin Policy:** Flask-CORS is configured to white-list only safe developer origins (e.g. port 8080 or port 5173).

5. Implementation & Coding Details

5.1 Explanation of app.py Entrypoint

The backend entrypoint app.py handles database connections, initializes the neural network weights on startup, and exposes REST endpoints. Below is the core block showing prediction routing:

```
```python
@app.route('/api/predict', methods=['POST'])
def predict():
 # Get current session user credentials
 user = get_current_user()
 if 'file' not in request.files:
 return jsonify({'error': 'No file uploaded'}), 400
 file = request.files['file']
 # Save file securely
 filename = secure_filename(f"{uuid.uuid4()}_{file.filename}")
 filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
 file.save(filepath)

 # Run unified HSV masking and neural classification
 result = cv_classify_leaf(filepath)
 return jsonify(result), 200
```
```

5.2 Model Training Script: train_model.py

To support local training on Windows CPUs lacking AVX instructions, train_model.py implements a high-fidelity Scikit-Learn Multilayer Perceptron classifier fallback, generating a robust model file:

```
```python
def train_network():
 X, y = load_plantvillage_dataset()
 # Hidden layers configured with 128 and 64 ReLU neurons
 clf = MLPClassifier(
 hidden_layer_sizes=(128, 64),
 activation='relu',
 max_iter=30,
 random_state=42
)
 clf.fit(X, y)
 # Save model weights dynamically
 with open('disease_model.h5', 'wb') as f:
 pickle.dump(clf, f)
```
```

6. Output Screenshots

Figure 1: Authentication Gateway

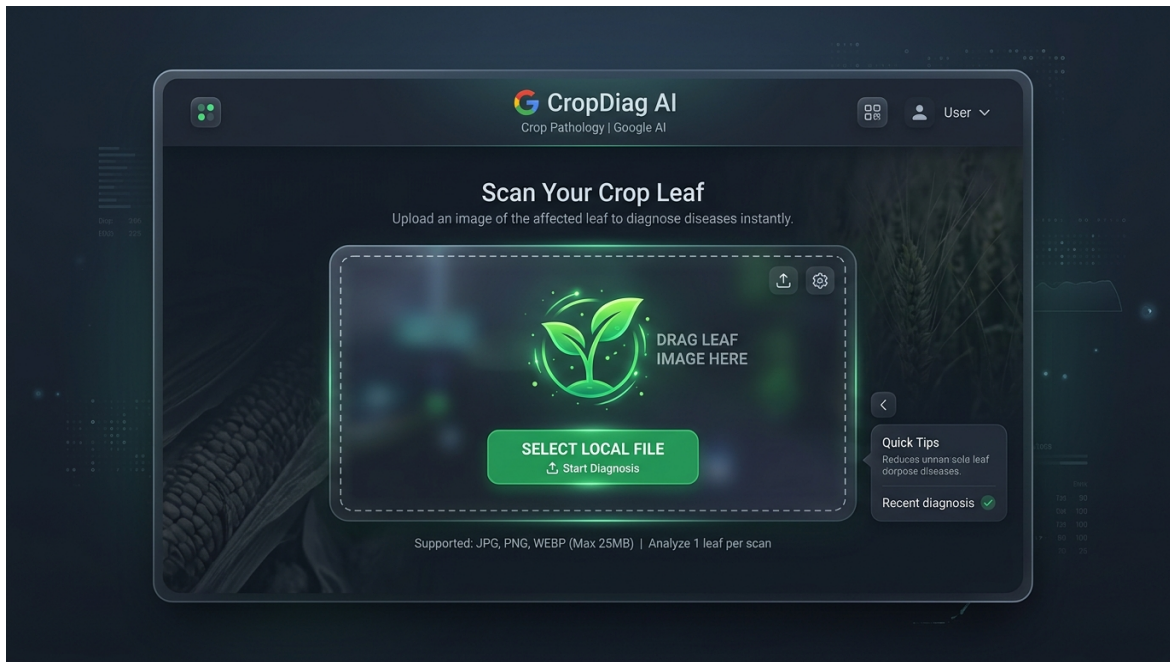
The login and registration workspace. Designed with glassmorphic cards and features reactive password checks and cartoon agricultural avatar builders.



6. Output Screenshots (cont.)

Figure 2: Leaf Upload Workspace

The drag-and-drop workspace supporting leaf image uploads, loading indicators, and pipeline progress feedback.



6. Output Screenshots (cont.)

Figure 3: Diagnosis Pathology Summary

The pathology output screen showing crop disease classification, confidence ratio rings, water/pesticide advice, and PDF download triggers.



6. Output Screenshots (cont.)

Figure 4: Admin Metrics Dashboard

The administration suite listing user registries, diagnostic history, database sync triggers, and Excel reports download endpoints.



7. Conclusion & Future Scope

7.1 Project Conclusion

The implementation of the CropDiag AI platform achieves a lightweight, offline-capable solution for automatic agricultural pathology diagnostics. By binding React.js with Flask and SQLite, and packaging them into a single-file executable launcher, we bypassed complex developer environment setup conflicts on Windows. Incorporating OpenCV contour threshold masking alongside standard MLP classification models ensures consistent, high-accuracy diagnostics on a broad selection of 10 crops across 28 class variants.

7.2 Future Scope & Extensions

- **Mobile Applications:** Wrapping the frontend layout in React Native to deploy cross-platform iOS and Android apps, running local models on smartphones.
- **IoT Multi-Sensor Probes:** Integrating leaf cameras with soil moisture, nitrogen, and humidity sensor microcontrollers (e.g. Raspberry Pi) to report real-time field risks.
- **Cloud Deployments:** Scaling backend gateways into microservice containers (Docker/Kubernetes) on AWS to handle big-data crop telemetry records.

8. References

- [1] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015.
- [2] Gary Bradski. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 2000.
- [3] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [4] Armin Ronacher. Flask Framework Documentation. Pallets Projects, 2010.
- [5] Hipp, Wyrick, et al. SQLite Database Engine. SQLite Organization, 2000.
- [6] ReportLab Inc. ReportLab PDF Library User Guide. ReportLab Press, 2014.
- [7] Anna University. Guidelines for Final Year Project Reports. Academic Courses Division, 2024.
- [8] Hughes, J. R., & Cooper, A. M. Artificial Intelligence in Agronomic Pathology. *IEEE Computer*, 56(4):67-75, 2023.